Tab 1



SE 801: Software Project Lab - III

SurveyClass: A Comprehensive Tool for Data Collection and Classification of Programming Problems by Difficulty Level

Submitted by

Monayem Sarker: BSSE 1228

Supervisor

Toukir Ahammed

Lecturer

Institute of Information Technology, University of Dhaka

Letter of Transmittal

1 December 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical report on “SurveyClass: A Comprehensive Tool for Data Collection and Classification of Programming Problems by Difficulty Level”.

Sir,
With due respect, I am pleased to submit the technical report of Software Project Lab 3 on “SurveyClass: A Comprehensive Tool for Data Collection and Classification of Programming Problems by Difficulty Level”. I have tried my best to deliver a good understanding about the technical details of the project. However, it might lack perfection. So, I, therefore, hope that you would be kind enough to accept my report and oblige thereby.

Sincerely yours,
Monayem Sarker
Roll: BSSE 1228
Institute of Information Technology
University of Dhaka

Supervisor’s Signature

Acknowledgment

I am grateful to the Institute of Information Technology for giving me such a tremendous opportunity to work on “SurveyClass: A Comprehensive Tool for Data Collection and Classification of Programming Problems by Difficulty Level”. I would like to convey my tremendous gratitude to my supervisor, Toukir Ahmed, Lecturer, Institute of Information Technology, University of Dhaka, and Alim Ul Gias, Lecturer in Computer Science (Education), City, University of London, for providing me guidelines about how I can prepare this analysis document.

Abstract

Determining the difficulty of programming problems is a critical task. Current methods often rely on subjective human judgment. This can be time consuming and inconsistent as well. To address this, we propose SurveyClass, a comprehensive system that uses human feedback and machine learning to classify programming problems objectively and efficiently.

SurveyClass is a combination of two systems. First, a survey platform collects pairwise comparisons of programming problems from human participants, which creates a dataset of perceived difficulty levels. This data is then processed using a pairwise ranking algorithm to generate problem difficulty. Second, a machine learning model is trained on this labeled dataset to learn the relationship between specific code features, such as loops and conditions, and problem difficulty.

By combining human expertise and machine learning, SurveyClass aims to provide a scalable solution for classifying programming problems.

Contents

1. Introduction **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Assumptions 1
2. Project Description **1**
2.1 Quality Function Deployment 1
2.2 Usage Scenario 1
2.2.1 Admin Functionality 2
2.2.2 Problem and Participant Management 2
2.2.3 Survey Management 2
2.2.4 Survey Participation 2
2.2.5 Classifying Problems 3
3. Scenario Based Modeling **3**
3.1 Use Case Diagram 3
3.1.1 SurveyClass 3
3.1.2 Modules of SurveyClass 3
3.1.2.1 Administration 3
3.1.2.1.1 Modules of Administration 4
3.1.2.1.1.1 Problems Management 4
3.1.2.1.1.2 Participants Management 5
3.1.2.1.1.3 Survey Management 5
3.1.2.2 Survey 5
3.1.2.3 Classification 5
4. Data Based Modeling **6**
5. Class Based Modeling **7**
5.1 Analysis Classes 7
5.2 Class Cards 8
5.3 Class Responsibility and Collaboration Diagram 11
6. Architectural Design **11**
6.1 Architectural Context Diagram 11
6.2 Define Archetypes 12
6.2.1 Pipe and Filter Architecture 12
6.2.2 Model-View-Controller (MVC) 14
6.3 Top Level Components 15
6.4 Instantiation of the Architecture 15

6.4.1 Component: Problem Management 15
6.4.2 Component: Participant Management 15
6.4.3 Component: Survey Management 16
6.4.4 Component: Survey 16
6.4.5 Component: Cassifier 16
7. Preliminary Test Plan 17
7.1 High Level Description of Testing Goals 17
7.2 Test Cases 17
8. Conclusion 22

Figures

Figure 1: Level 0 Use Case 4

Figure 2: Level 1 Use Case 5

Figure 3: Level 1.1 Use Case 6

Figure 4: ER-Diagram of SurveyClass 7

Figure 5: Class Responsibility and Collaboration Diagram 13

Figure 6: Architectural Context Diagram 14

Figure 7: Top Level Components 15

Figure 8: Pipe and Filter of Problem Management 16

Figure 9: Pipe and Filter of Participant Management 16

Figure 10: Pipe and Filter of Survey Management 17

Figure 11: MVC of Survey 17

Figure 12: Pipe and Filter of Classifier 18

1. Introduction

SurveyClass is a comprehensive tool to classify programming problems based on difficulty. It is a two-part system. The first part is a survey platform, where participants are presented with pairs of problems to rank based on perceived difficulty. This input feeds into a pairwise ranking algorithm which responses to generate difficulty levels. The second part of the system is a machine learning model that uses survey results as labels, mapping specific code features such as loops and conditions to difficulty levels. Over time, this model will be capable of automatically classifying new problems without further human input.

The primary objective of SurveyClass is to develop an integrated platform for data collection and classification.

1.1 Purpose

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

1.2 Scope

The scope of the project is given below-

- Web Application Development for the survey tool
- Pairwise Ranking Algorithm Implementation
- Machine Learning Model Development for classification

1.3 Assumptions

The assumptions of the project are-

- The source code will not have any compilation errors
- This may not accurately classify the problem as accuracy is not the focus

2. Project Description

This chapter presents the quality function deployment and usage scenario of the tool.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified-

Normal Requirements

- Admin can log in and create additional admins.
- Admin can create/manage problem sets, add problems individually or in bulk.
- Admin can register participants individually or in bulk.
- Admin can create surveys and send invitation emails.
- Admin can search surveys and check details.
- Participants receive survey invitations via email with a direct link.
- Participants can start surveys easily with no duplicate comparisons.
- End users can upload single or multiple problems for classification.
- End users can download classification results in CSV format.
- System ranks problems using a pairwise comparison algorithm.

Expected Requirements

- System supports bulk uploads for problems and participants.
- The system suggests optimized pairs for comparison to reduce redundant comparisons.
- Reminders for participants who haven't completed surveys.

Exciting Requirements

- Admins can see the survey stats.

2.2 Usage Scenario

The “SurveyClass: A Comprehensive Tool for Data Collection and Classification of Programming Problems by Difficulty Level” will be a web application. Usage scenario for the app has been segmented and described below:

2.2.1 Admin Functionality

Admins can access the platform using authentication, add participants who will be participating in the survey, add programming problems to the survey, and manage survey settings, reminders. Admins can also view survey results.

2.2.2 Problem and Participant Management

Admins can create programming problems, as well as register and manage participant profiles. Problems can be added individually or in bulk, and can be uploaded under problem sets. Participant information, such as name, email address, and other relevant details, can be added individually or in bulk.

2.2.3 Survey Management

Admins can create surveys by selecting one or more problem sets. Once created, the system generates unique invitation keys for each participant, which are sent via email. Admins can monitor survey progress, track completion rates. To encourage participation, admins can send reminder emails to these participants.

2.2.4 Survey Participation

Participants receive unique invitation keys via email. Upon receiving the key, they can access the survey platform and authenticate themselves using the key. Once verified, participants are presented with pairwise comparisons of programming problems. They are asked to select the most difficult problem in each pair. Each participant is typically required to complete 20 such comparisons per survey.

2.2.5 Classifying Problems

The system uses a machine learning model to automatically classify the difficulty of Python programming problems. This model is trained on a dataset of problems that have been manually rated by human experts. By analyzing various code features, the system can accurately predict the difficulty level of new problems.

3. Scenario Based Modeling

This chapter describes the scenario based modeling for the program slicing tool.

3.1 Use Case Diagram

Use case diagrams are given below-

3.1.1 SurveyClass

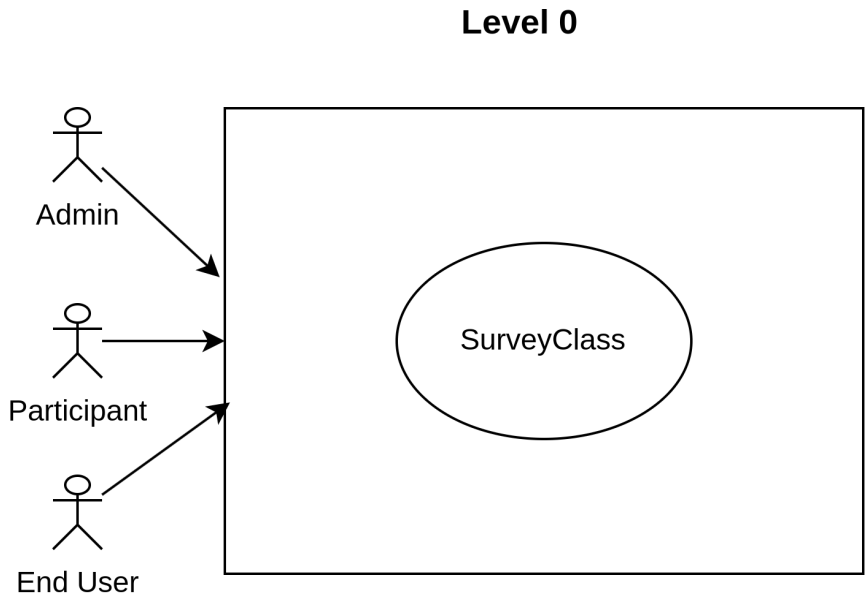


Figure 1: Level 0 Use Case

Primary actors: Admin, Survey Participant, End User

Secondary actors: There is no secondary actor

Goal in context: The diagram shown above represents the whole SurveyClass.

3.1.2 Modules of SurveyClass

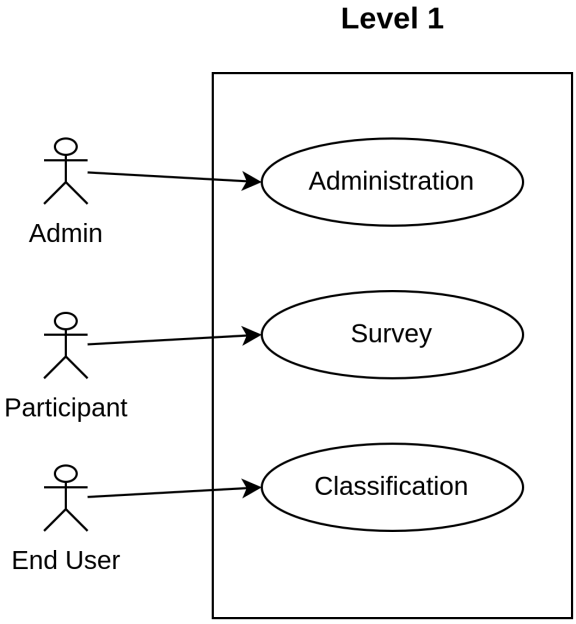


Figure 2: Level 1 Use Case

Primary actors: Admin, Survey Participant, End User

Secondary actors: There is no secondary actor

Goal in context: The diagram shown above shows all the modules of SurveyClass.

The “SurveyClass” consists of 3 modules. They are-

3.1.2.1 Administration

The Administration module handles all core admin functionalities, allowing the admin to manage every aspect of the system. Through this module, the admin can perform problem management by creating and organizing programming problems under specific problem sets. It also facilitates participant management, enabling the admin to create and manage participant profiles for surveys. Additionally, the module includes survey management, allowing the admin to create surveys linked to one or more problem sets, send invitations to participants, and analyze survey statistics to gain insights into survey performance and results.

3.1.2.1.1 Modules of Administration

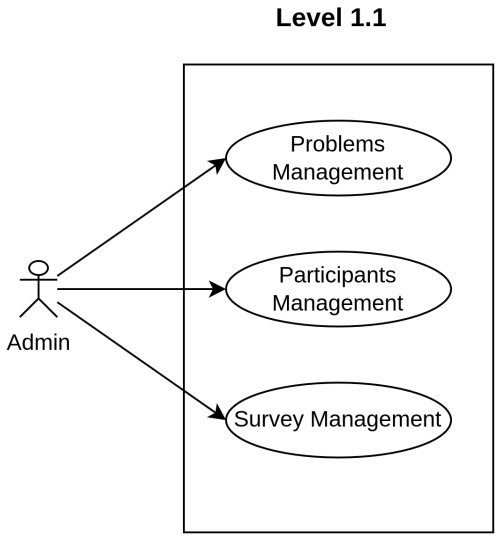


Figure 3: Level 1.1 Use Case

Primary actors: Admin
Secondary actors: There is no secondary actor
Goal in context: The diagram shown above shows the sub modules of Administration .

3.1.2.1.1.1 Problems Management
This module allows the admin to create programming problems under a problem set.

3.1.2.1.1.2 Participants Management
This module enables the admin to create participants.

3.1.2.1.1.3 Survey Management
This module allows the admin to create surveys with one or more problem sets. Admins can send invitations to participants, and analyze survey stats.

3.1.2.2 Survey

This module manages the survey process. Invitations are sent to participants with a unique survey key. Participants must validate the key through the web application to access the survey. Once validated, participants will complete the survey by performing 20 pairwise comparisons between programming problems presented in pairs. Each comparison involves selecting the more difficult problem between the two. The results of these comparisons are recorded and analyzed for further processing.

3.1.2.3 Classification

This module leverages a machine learning model trained on the surveyed data to classify programming problems. The surveyed data, gathered through pairwise comparisons, serves as the training dataset for the model. Once trained, the model is used to predict and classify problems into predefined difficulty categories (e.g., easy, medium, hard) based on their characteristics and the feedback obtained from the survey.

4. Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

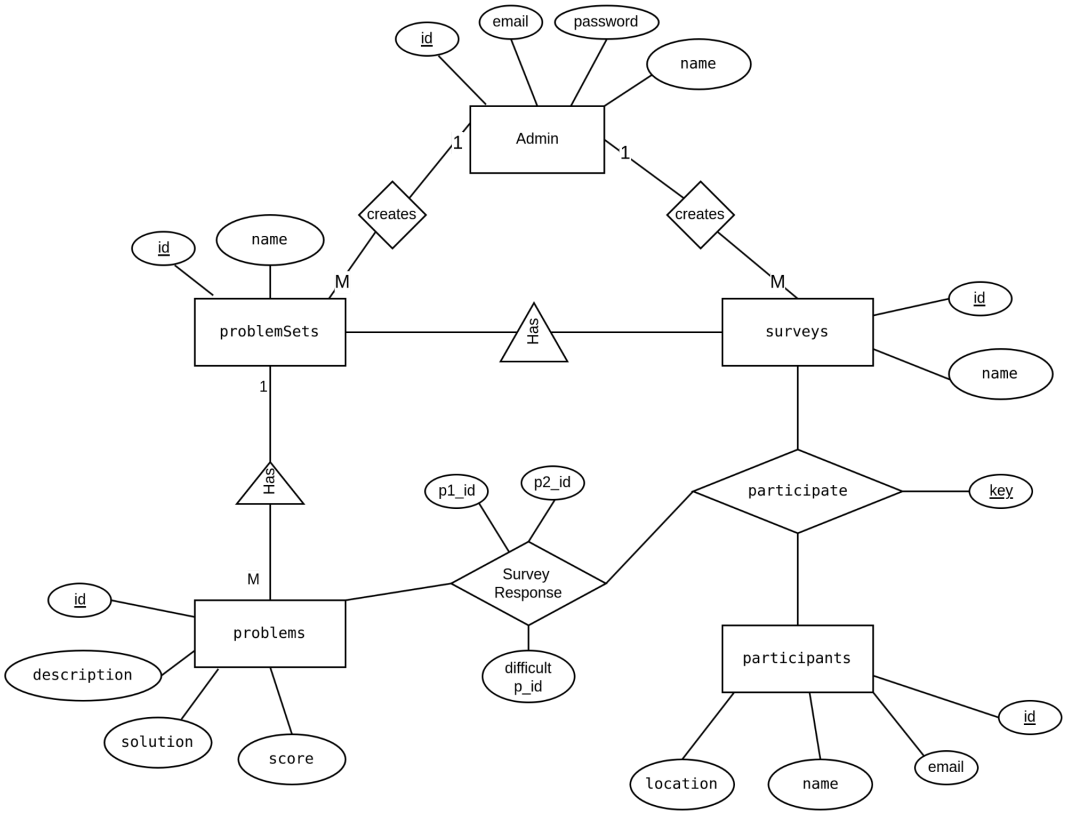


Figure 4: ER-Diagram of SurveyClass

The schemas derived from the ER-diagram is as follows:

Schema	Attributes	Type
Admin	id (PK)	varchar
	email	varchar
	password	varchar
	name	varchar
ProblemSets	id (PK)	varchar
	name	varchar
Problems	id (PK)	varchar
	description	text
	code	text
	score	int
	problemSet_id (FK)	varchar
Surveys	id (PK)	varchar
	name	varchar
	problemSet_ids(FK)	varchar[]
Survey Response	Difficult_problem_id	varchar
	problem1_id (FK)	varchar
	problem2_id (FK)	varchar
Participants	id (PK)	varchar
	name	varchar
	email	varchar
	location	text

Participate	code(PK)	varchar
	survey_id (FK)	varchar
	participant_id (FK)	varchar

Table-1: Schemas of SurveyClass

5. Class Based Modeling

This chapter describes the class based modeling of the SurveyClass.

5.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (Administration, Admin, Problem, Problem Set, Participant, Survey, Survey Key, Pairwise Comparison, Programming Problem, Machine Learning Model, Difficulty Category, Survey Results, Survey). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes-

- Admin
- Problem
- ProblemSet
- Participant
- Survey
- Comparison
- SurveyResults
- Classifier

5.2 Class Cards

Class cards for this tool are documented below-

Admin	
Attributes	**Methods**
- Name - UserName - Password - Email	- createProblemSet() - manageParticipants() - createSurvey() - analyzeSurveyStats()
Responsibilities	**Collaborators**
- Manage problems. - Manage participants. - Create and manage surveys	- ProblemSet - Participant - Survey

Problem	
Attributes	**Methods**
- ID - Description - Code - Difficulty Level (enum: easy, medium, hard) - ProblemSet_id	- createProblem() - createProblemInBulk()
Responsibilities	**Collaborators**
- Represent an individual programming problem.	- ProblemSet

ProblemSet	
Attributes	**Methods**
- ID - Name - List of Problems (List of Problem objects)e	- addProblem()
Responsibilities	**Collaborators**
- Manage a collection of programming problems.	- Problem - Survey

Participant	
Attributes	**Methods**
- ID - Name - Email - Location - Designation	- validateSurveyKey() - participateInSurvey()
Responsibilities	**Collaborators**
- Represent participants involved in the survey.	- Survey

Survey	
Attributes	**Methods**
- ID - Problem Set List - Participant List - Survey Key - Survey Results	- sendInvitation() - validateKey() - recordComparisonResults()

Responsibilities	Collaborators
- Manage the survey process - Send survey invitations - Collect pairwise comparisons.	- ProblemSet - Participant - SurveyResults

Comparison	
Attributes	**Methods**
- Problem 1 - Problem 2 - Selected Problem	- storeComparison()
Responsibilities	**Collaborators**
- Represent an individual pairwise comparison between two problems.	- Problem

SurveyResults	
Attributes	**Methods**
- ID - Participant - Pairwise Comparisons	- saveResults() - fetchResults()
Responsibilities	**Collaborators**
- Handle and store survey results, including pairwise comparisons.	- Participant - Comparison

Classifier	
Attributes	**Methods**
- Model - Training Data - Predicted Categories	- trainModel() - predictDifficulty()
Responsibilities	**Collaborators**
- Use machine learning to classify programming problems based on survey data.	- Problem - SurveyResults

5.3 Class Responsibility and Collaboration Diagram

The CRC diagram for SurveyClass is given below-

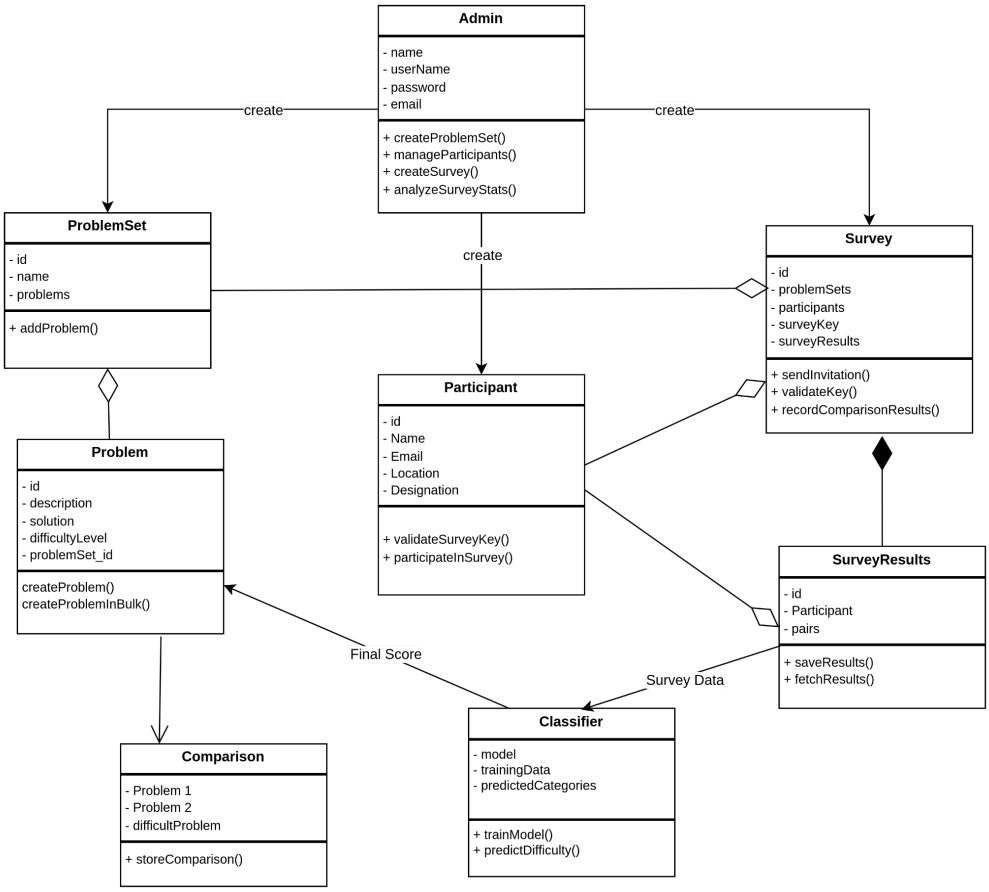


Figure 5: Class Responsibility and Collaboration Diagram

6. Architectural Design

This chapter describes the architectural context diagram, archetypes and top level components for the tool.

6.1 Architectural Context Diagram

The architectural context diagram for the tool is given below-

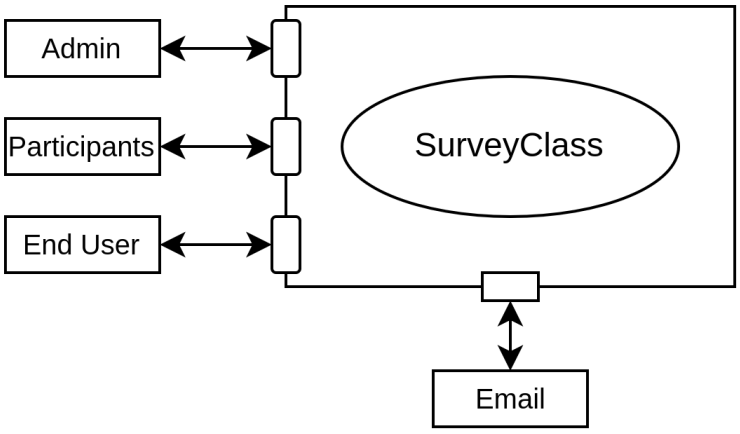


Figure 6: Architectural Context Diagram

Three categories of users are supposed to be the target for the tool. They are admin, survey participant, end user. All types of users are the producers and also consumers of the information through the tool. The tool uses email as a subordinate system for survey invitation. There are no superordinate or peer-level systems.

6.2 Define Archetypes

The FindYourBug system uses the following archetypes:

1. Pipe and filter
2. Model-View-Controller (MVC)

6.2.1 Pipe and Filter Architecture

The Pipe and Filter architecture is a software design pattern that organizes a system as a series of processing elements (filters) connected by communication channels (pipes). It is particularly useful for designing applications that involve processing streams of data in a linear and sequential manner. Each filter performs a specific task on the incoming data and passes the results to the next filter through the pipes, creating a data flow pipeline.

1. **Filter:** All processes that contain a significant general algorithm that can be reused in other tasks. The filters can be re-ordered to solve different problems.
2. **Pipe:** Specific tasks that are required to communicate between the filters.

6.2.2 Model-View-Controller (MVC)

The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller. Each architectural component is built to handle specific development aspects of an application. It isolates the business logic and presentation layer from each other. It was traditionally used for desktop graphical user interfaces (GUIs).
Model: The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.
View: The View component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller.
Controller: The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.

6.3 Top Level Components

The overall architecture with top level components are illustrated below. Note that the top level components show a very high level view of our system. Each component has independent responsibilities.

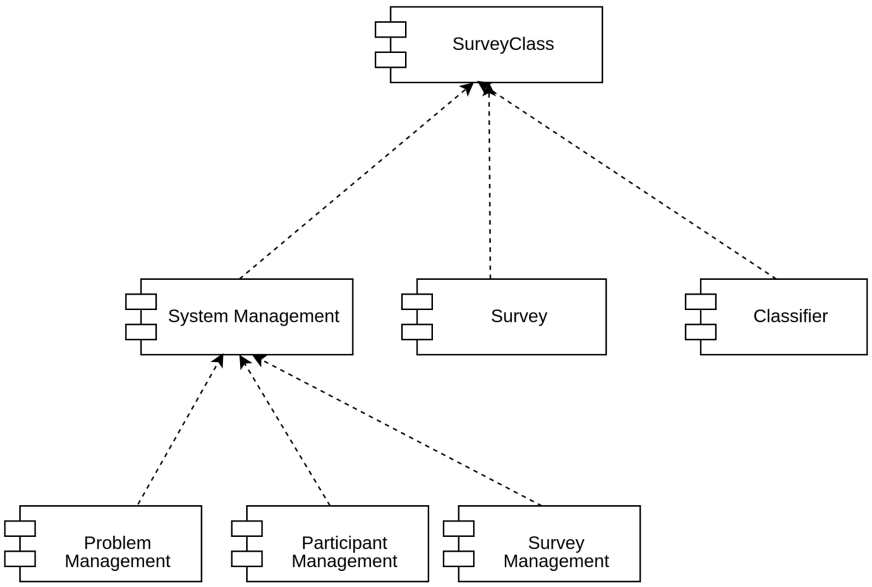


Figure 7: Top Level Components

6.4 Instantiation of the Architecture

The components identified in Section - 6.3 are broad. So they are refined and instantiated into smaller components. We use different architectural patterns which better suite each component

6.4.1 Component: Problem Management

Pattern: Pipe and Filter



Figure 8: Pipe and Filter of Problem Management

The Problem Management component utilizes the Pipe and Filter pattern to streamline the handling of programming problems. The pipeline begins with the Input Filter, which parses and validates problem details, including their description, code, and difficulty level. The validated data then moves to the Processing Filter, where problems are either added to the problem sets. Finally, the Output Filter saves the processed problems into the database and makes them available for use in surveys, ensuring efficient and consistent management of problems.

6.4.2 Component: Participant Management

Pattern: Pipe and Filter



Figure 9: Pipe and Filter of Participant Management

The Participant Management component applies the Pipe and Filter pattern to handle participant-related operations effectively. The pipeline starts with the Input Filter, which parses participant details. Next, the Processing Filter validates these details to ensure a participant. In the last stage, the Output Filter saves participant data in the system.

6.4.3 Component: Survey Management

Pattern: Pipe and Filter

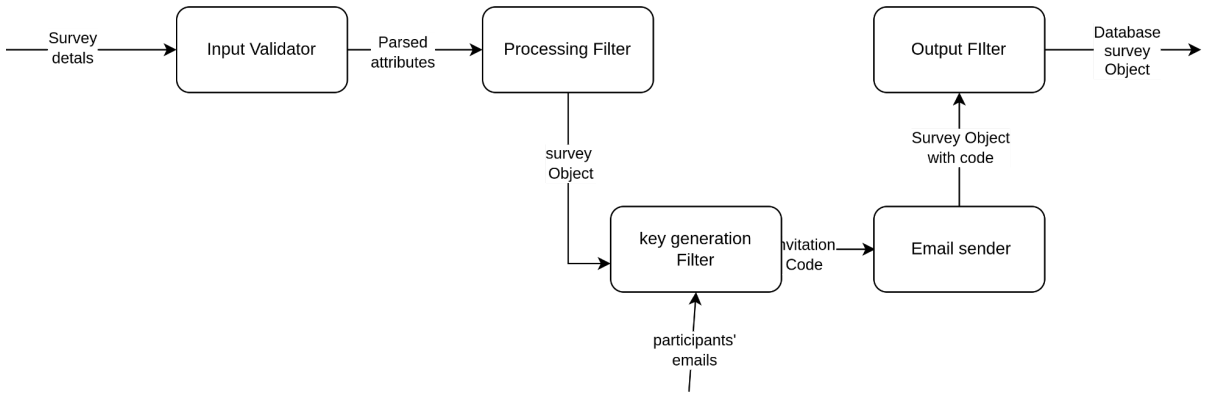


Figure 10: Pipe and Filter of Survey Management

The survey component is best suited for MVC patterns. The pipeline starts with the Input Filter, which parses participant details. Next, the Processing Filter validates these details to ensure a participant. In the last stage, the Output Filter saves participant data in the system.

6.4.4 Component: Survey

Pattern: Pipe and Filter

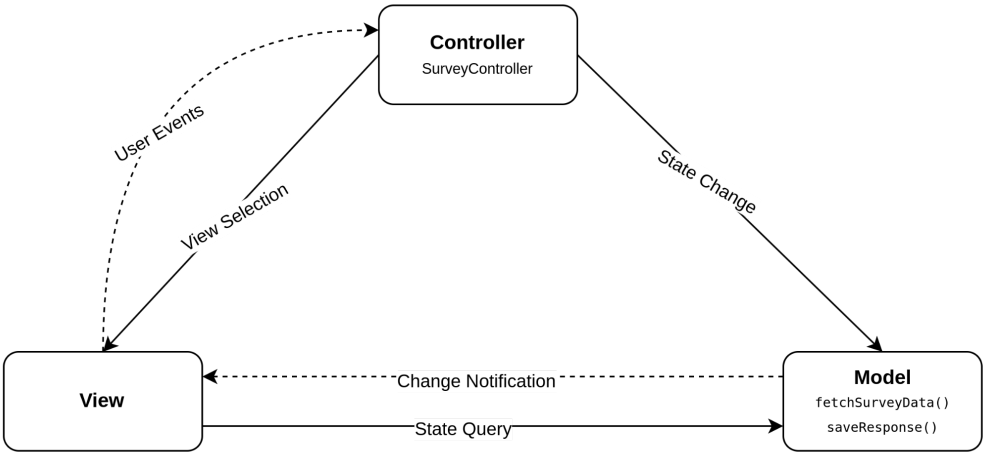


Figure 11: MVC of Survey

The survey component is best suited for the MVC (Model-View-Controller) pattern. In this approach, the View provides a user-friendly interface where participants select the more difficult problem from the presented pair. The Controller handles the user interactions, processes the selected choice, and ensures the input is valid. Finally, the Model saves the participant's selection directly to the database, maintaining a clear separation of concerns and enabling efficient data management.

6.4.5 **Component:** Cassifier

Pattern: Pipe and Filter



Figure 12: Pipe and Filter of Classifier

The Classifier component applies the Pipe and Filter pattern to handle classification of problems. The pipeline starts with the Input Filter, which parses new problem description and solution. Next, the Feature Extraction Filter validates these details to normalize the data. In the last stage, the Machine learning Filter runs the model and output a difficulty score for the problem.

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To demonstrate that the tool meets its functional and non-functional requirements.
- To ensure surveys are created, managed, and executed properly.
- To validate that problem data and participant responses are processed accurately.
- To ensure that problem difficulty is classified correctly based on participant responses.
- To identify and eliminate any bugs in the system.
- To ensure the tool runs smoothly and efficiently under various conditions.

7.2 Test Cases

Test cases for the testing of this tool are documented below-

Test ID: T1

Test Case: Test if participant details are parsed properly.

Input Test Data: JSON file containing participant details.

Steps To Be Executed:

1. Run the tool
2. Verify parsed participant details.

Expected Result: TParticipant details should match the data in the input file.

Target Item: Participant Management Module.

Pass/Fail: Pending

Test ID: T2

Test Case: Test if problems are uploaded correctly.

Input Test Data: CSV file containing problem descriptions and details

Steps To Be Executed:

1. Upload the problem data using the tool
2. Verify the database entries for problems

Expected Result: The database should contain all problems as specified in the input CSV.

Target Item: Problem Management Module.

Pass/Fail: Pending

Test ID: T3

Test Case: Test if problem pairs are generated properly for surveys

Input Test Data: List of problems and survey configuration.

Steps To Be Executed:

1. Configure a survey using the tool.
2. Verify the generated problem pairs.

Expected Result: The problem pairs should meet the survey configuration criteria.

Target Item: Survey Management Module.

Pass/Fail: Pending

Test ID: T4

Test Case: Test if participants' responses are recorded accurately

Input Test Data: Participant responses to problem pairs.

Steps To Be Executed:

1. Submit survey responses using the tool.
2. Check the database for saved responses.

Expected Result: Responses should be recorded accurately in the database.

Target Item: Response Recording Module.

Pass/Fail: Pending

Test ID: T5

Test Case: Test if difficulty scores are calculated correctly.

Input Test Data: Participant responses and survey data.

Steps To Be Executed:

1. Run the difficulty classification tool.
2. Verify the generated difficulty scores.

Expected Result: Difficulty scores should accurately reflect the responses and survey data.

Target Item: Difficulty Classifier.

Pass/Fail: Pending

Test ID: T6

Test Case: Test if survey results are displayed correctly.

Input Test Data: Survey data and difficulty classification results.

Steps To Be Executed:

1. Access the survey results through the tool.l
2. Verify the displayed information.

Expected Result: Survey results should be clear, accurate, and comprehensible.

Target Item: Survey Result Viewer.

Pass/Fail: Pending

Test ID: T7

Test Case: Test if surveys are managed properly

Input Test Data: Survey configurations and problem sets

Steps To Be Executed:

1. Create surveys using the tool.
2. Verify the changes in the database

Expected Result: Surveys should be created.

Target Item: Survey Management Module.

Pass/Fail: Pending

Test ID: T8

Test Case: Test if authentication and authorization work correctly.

Input Test Data: Admin and participant credentials.

Steps To Be Executed:

1. Log in as an admin and perform survey operations.
2. og in as a participant and access surveys.

Expected Result: Admins should have full access, and participants should have restricted access

Target Item: Authentication Module

Pass/Fail: Pending

Test ID: T9

Test Case: Test if survey invitation keys are validated correctly.

Input Test Data: JValid and invalid survey invitation keys.

Steps To Be Executed:

1. Attempt to access surveys using the keys.
2. Verify the system's response.

Expected Result: Valid keys should grant access, and invalid keys should be rejected.

Target Item: Survey Access Validator.

Pass/Fail: Pending

Test ID: T10

Test Case: Test the system's performance under heavy load.

Input Test Data: Large sets of problems and participants.

Steps To Be Executed:

1. Upload large datasets and execute multiple surveys simultaneously.
2. Monitor system performance and response times.

Expected Result: The system should handle the load smoothly without crashes or significant delays.

Target Item: Overall System Performance.

Pass/Fail: Pending

8. Conclusion

In this report, I have discussed all the necessary technical details to develop the “SurveyClass: A Comprehensive Tool for Data Collection and Classification of Programming Problems by Difficulty Level”. I have discussed the scope, assumptions, QFD and usage scenario. Scenario-based modeling, data modeling and class-based modeling have been performed. The architectural design has been discussed. I have also presented the high-level description of testing goals and summary of items and features to be tested. With this document, I have tried to clearly state the workflow of the total system. I hope this report can help the reader have an appropriate understanding of the technical details of this tool.